

Laboratory 03: Inter-Process Communication Using Signals, FIFO, and Message Passing

1. Laboratory Title

Inter-Process Communication Using Signals, FIFO, and System V Message Queues

2. General Objective

Develop a multi-process communication mechanism in Linux using signals, FIFO, and System V Message Queues, allowing three independent processes to exchange information through a defined communication sequence.

3. Specific Objectives

At the end of the laboratory, the student will be able to:

1. Use Linux signals to trigger specific actions in a running process.
2. Implement and verify inter-process communication using a FIFO and System V Message Queues.
3. Analyze the flow of information between multiple processes using different IPC mechanisms.

4. Resources and Required Tools

- Linux operating system.
- Linux terminal.
- GCC or G++ compiler.
- C or C++ programming language.
- Source code provided by the professor.
- Three terminals for executing the processes.
- Basic knowledge of:
 - Processes.
 - Process identifiers (PID).
 - Linux signals.
 - FIFO / named pipes.
 - System V IPC.
 - Message Queues.
 - Compilation and execution of C/C++ programs.

Important: The source code for the programs will be provided by the professor. Students must modify, complete, integrate, or test the provided code according to the requirements of this laboratory.

5. Brief Introduction

Operating systems provide several mechanisms that allow processes to communicate and coordinate their activities.

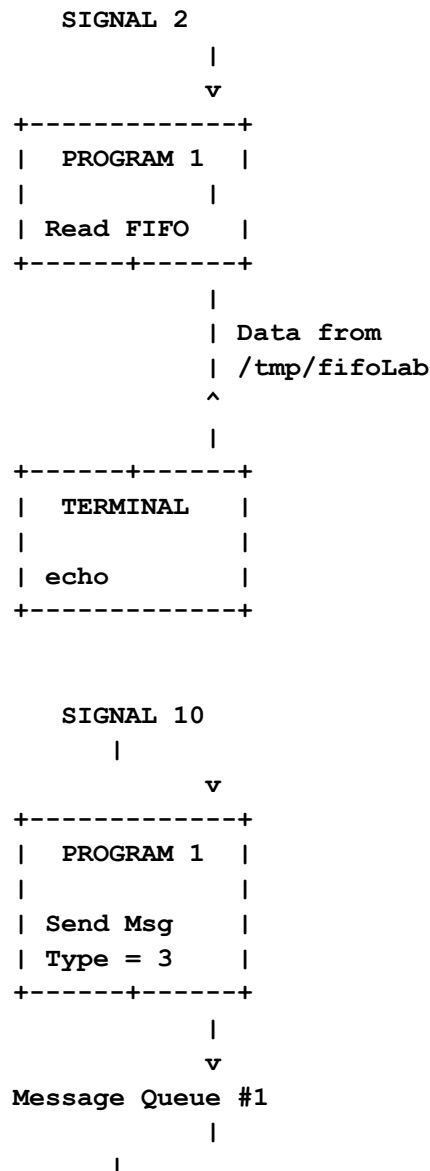
In this laboratory, students will work with three IPC mechanisms:

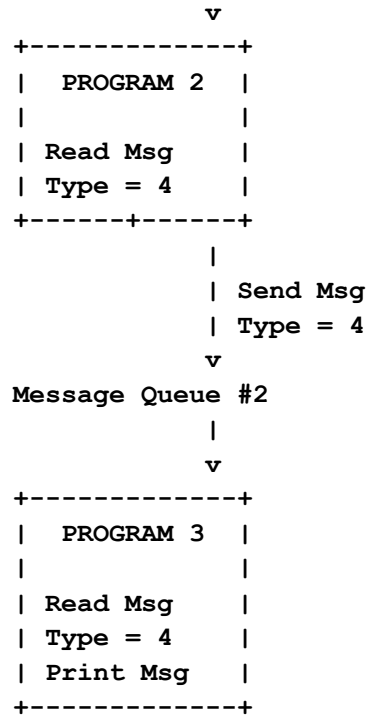
- Signals: used to notify a process that a specific action must be performed.
- FIFO (Named Pipe): used to transfer data between processes through the special file `/tmp/fifoLab`.
- System V Message Queues: used to exchange structured messages between processes.

The laboratory consists of three programs:

- Program 1: receives two different signals. One signal activates reading from the FIFO, while the other activates message transmission to Program 2.
- Program 2: receives a message from Program 1 and forwards it to Program 3 using a different Message Queue and a different message type.
- Program 3: receives the message and displays it on the terminal.

The communication architecture is:





Each program must provide debugging output whenever it sends or receives data. This output will allow the student to verify the communication sequence during execution.

6. Laboratory Instructions

Step 1 — Prepare the environment

Obtain the source code for the three programs from the professor.

Identify:

- Program 1.
- Program 2.
- Program 3.
- The FIFO `/tmp/fifoLab`.
- The Message Queue keys used by the programs.

Do not create an alternative architecture. Follow the communication structure specified in this laboratory.

Step 2 — Compile the programs

Compile the three programs using the appropriate C or C++ compiler.

Verify that:

- All programs compile successfully.
- No compilation errors remain.
- The three executable files can be launched from the terminal.

The programs must be executed as independent processes.

Step 3 — Prepare the FIFO

Verify that the FIFO used by the laboratory is:

```
/tmp/fifoLab
```

If necessary, create the FIFO following the instructions provided by the professor.

The terminal used for writing data to the FIFO must use the `echo` command.

For example, the student may use a command following this format:

```
echo "Test message" > /tmp/fifoLab
```

The command above is an operational example, not a solution to the laboratory.

Step 4 — Start Program 3

Execute Program 3 in a terminal.

Program 3 must remain active and wait for a message from Program 2.

It must use the Message Queue associated with the second communication channel and wait specifically for message type 4.

When Program 3 receives data, it must print the received message in the terminal.

Program 3 must also display a debugging message indicating that data has been received.

Step 5 — Start Program 2

Execute Program 2 in a second terminal.

Program 2 must wait for a message sent by Program 1.

The message received from Program 1 must have:

- The Message Queue associated with Program 1 → Program 2.
- Message type 3.

When Program 2 receives the message, it must:

1. Display a debugging message indicating that data was received.
2. Process the received message.
3. Forward the message to Program 3.
4. Use a different Message Queue key for the second communication.
5. Send the message using message type 4.
6. Display a debugging message indicating that the message was sent.

Step 6 — Start Program 1

Execute Program 1 in a third terminal.

Program 1 must remain active and wait for two different signals.

The required behavior is:

Signal	Required action
Signal 2	Read data from <code>/tmp/fifoLab</code>

Signal 10 Send a message to Program 2 using Message Queue, type

3

Program 1 must display debugging information when it:

- Receives Signal 2.
- Reads data from the FIFO.
- Receives Signal 10.
- Sends data to Program 2.

Step 7 — Test Signal 2 and FIFO communication

From another terminal, send a message to the FIFO using `echo`.

After writing the message to `/tmp/fifoLab`, send Signal 2 to Program 1.

Verify that Program 1:

1. Receives Signal 2.
2. Activates the FIFO-reading operation.
3. Reads the data from `/tmp/fifoLab`.
4. Displays appropriate debugging information.

Record the message displayed by Program 1.

Step 8 — Test Signal 10 and Message Passing

Use Signal 10 to activate the message transmission operation in Program 1.

Verify that:

1. Program 1 receives Signal 10.
2. Program 1 sends the message to Program 2.
3. The message uses message type 3.
4. Program 2 receives the message.
5. Program 2 displays the received message.
6. Program 2 sends the message to Program 3 using the second Message Queue.
7. The message sent to Program 3 uses message type 4.
8. Program 3 receives and displays the message.

Step 9 — Verify debugging information

Verify that every program provides debugging information for data transmission and reception.

The execution should allow the student to identify at least the following events:

Program 1: Signal 2 received

Program 1: Data read from FIFO

Program 1: Signal 10 received

Program 1: Message sent, type 3

Program 2: Message received, type 3

Program 2: Message sent, type 4

Program 3: Message received, type 4

Program 3: Message displayed

The exact wording of the debugging messages is left to the implementation, but the information must be sufficient to identify the operation being performed.

Step 10 — Complete the communication test

Perform a complete test of the system.

Demonstrate the following sequence:

```
Terminal
|
| echo
v
/tmp/fifoLab
|
| Signal 2
v
Program 1
|
| Signal 10
v
Message Queue #1
Type 3
|
v
Program 2
|
| Message Queue #2
| Type 4
v
Program 3
|
v
Terminal
```

Verify that the message successfully travels through the complete communication chain.

7. Practical Exercises

Exercise 1 — Signal and FIFO

Send a message to `/tmp/fifoLab` using the `echo` command.

Then activate Program 1 using Signal 2.

Verify and explain:

1. What process receives Signal 2?
2. What operation is triggered by the signal?
3. From where does Program 1 obtain the data?
4. What debugging information is displayed?

Exercise 2 — Signal and Message Queue

Activate Program 1 using Signal 10.

Verify that the message is sent to Program 2 using message type 3.

Answer:

1. Which process sends the message?
2. Which process receives it?
3. What Message Queue is used?
4. What message type is used?
5. How can the debugging output confirm that the transmission was successful?

Exercise 3 — Program 2 to Program 3

Verify that Program 2 forwards the received message to Program 3.

The second communication must use:

- A different Message Queue key.
- Message type 4.

Answer:

1. Why does Program 2 need to receive the message before forwarding it?
2. Why is a different Message Queue used?
3. What is the purpose of message type 4?
4. How does Program 3 verify that it received the message correctly?

Exercise 4 — Complete IPC Chain

Perform a complete execution of the three programs.

Document the following events:

Event	Process	IPC mechanism	Type/Signal
Data written	Terminal	FIFO	/tmp/fifoLab
FIFO activation	Program 1	Signal	2
FIFO reading	Program 1	FIFO	—
Message transmission	Program 1	Message Queue	Type 3
Message reception	Program 2	Message Queue	Type 3
Message transmission	Program 2	Message Queue	Type 4
Message reception	Program 3	Message Queue	Type 4

Finally, explain in your own words how signals, FIFO, and Message Queues work together in this laboratory.

8. Evaluation Criteria

Evaluation Criterion	Weight
Correct handling of Signal 2 and FIFO reading	20%
Correct handling of Signal 10	15%
Correct Program 1 → Program 2 communication using Message Queue and type 3	20%
Correct Program 2 → Program 3 communication using a different key and type 4	20%
Correct debugging information for send/receive operations	10%
Complete integration and demonstration of the three programs	10%
Code organization and good programming practices	5%
Total	100%

9. Deliverables

The student must submit two source-code files, using either C or C++:

1. Program 1
2. Program 2

The files may therefore be submitted as:

- `program1.c` and `program2.c`, or
- `program1.cpp` and `program2.cpp`.

Submission requirements

The submitted programs must:

- Compile without errors.
- Execute correctly under Linux.
- Implement the required signal handling.
- Use `/tmp/fifoLab` as the FIFO specified by the laboratory.
- Send the message from Program 1 to Program 2 using message type 3.
- Forward the message from Program 2 to Program 3 using a different Message Queue key and message type 4.
- Include appropriate debugging output when sending and receiving data.
- Follow good programming practices.
- Not include unnecessary functionality.

Program 3 is provided by the professor and is not included in the two source files that the student must submit.

The final demonstration must show the complete communication flow:

Terminal → FIFO → Program 1 → Message Queue (Type 3) → Program 2 → Message Queue (Type 4) → Program 3 → Terminal.